

VL-JEPA Learning Path: Paper Summaries & Reading List

For: Suhail | Path to Building VL-JEPA from Scratch

How to Use This Document

Read these papers in order. Each one builds on the last. Don't try to understand every equation — focus on the *idea*. The "Plain English Summary" for each paper is what actually matters. Read the real paper after you understand the idea.

Estimated total reading time: 6–10 hours across 2 weeks **Papers:** 5 core + 2 bonus **Goal:** By the end, you understand every component of VL-JEPA before writing a line of code

Paper 1: Attention Is All You Need (2017)

Authors: Vaswani et al. (Google Brain) **Link:** <https://arxiv.org/abs/1706.03762> **Read:** Sections 1, 2, 3, and Figure 1 only. Skip the machine translation experiments. **Time:** 30–45 minutes

Why You Need This

Every model in this list — ViT, CLIP, JEPA — is built on the Transformer. If you don't understand attention, nothing else will make sense. This is the foundation of modern AI.

Plain English Summary

The problem it solved: Before 2017, the best models for language were Recurrent Neural Networks (RNNs). They processed words one at a time, left to right. Problem: by the time you reached the end of a long sentence, the model had "forgotten" the beginning. Like reading a book but forgetting chapter 1 by chapter 10.

The big idea — Self-Attention: Instead of processing one word at a time, look at ALL words simultaneously and let each word "attend to" every other word based on relevance.

Example: "The animal didn't cross the street because it was too tired." - What does "it" refer to? The animal. - Self-attention figures this out by letting "it" attend strongly to "animal" and weakly to "street."

How attention works (simplified): For each word, you create three vectors: - **Query (Q):** "What am I looking for?" - **Key (K):** "What do I contain?" - **Value (V):** "What information do I pass along?"

Attention score = $Q \times K$ (dot product) Higher score = more attention = more information flows

Then: Output = weighted sum of all Values, weighted by attention scores

Why it matters for VL-JEPA: The Transformer's attention mechanism is the engine inside: - The Vision Encoder (reads image patches) - The Language Encoder (reads text tokens) - The Predictor (predicts missing representations)

What changed after this paper: Every major AI model since 2017 — GPT, BERT, Claude, ViT, CLIP, DALL-E, JEPA — is built on this architecture. It's the most important paper in modern AI history.

Key Equations (Don't Fear Them)

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) × V
```

Where:

- Q = query matrix
- K = key matrix
- V = value matrix
- d_k = dimension of keys (scaling factor to prevent vanishing gradients)
- softmax = converts scores to probabilities (sum to 1)

Mental model: Imagine a library. Q is your search query. K is the title of each book. V is the content inside each book. You

search for relevant books (Q-K), then read the relevant ones (weighted V).

Paper 2: An Image is Worth 16x16 Words (ViT, 2020)

Authors: Dosovitskiy et al. (Google Brain) **Link:** <https://arxiv.org/abs/2010.11929> **Read:** Sections 1, 2, 3 and Figure 1. Skip sections 4–6 (experiments). **Time:** 30 minutes

Why You Need This

VL-JEPA's vision encoder IS a Vision Transformer. You need to understand how images are turned into sequences that a Transformer can process. This is the bridge between pixels and attention.

Plain English Summary

The problem it solved: Before ViT, images were processed by Convolutional Neural Networks (CNNs). CNNs are good but have inductive biases — they assume nearby pixels matter more than distant ones. Transformers make no such assumption — but transformers need sequences, not 2D grids. How do you feed an image to a Transformer?

The big idea — Patch Embeddings: Cut the image into small fixed-size patches. Treat each patch like a word. Feed the sequence of patches into a standard Transformer.

Step by step: 1. Take a 224×224 image 2. Cut it into 16×16 pixel patches → you get 196 patches (14×14 grid) 3. Flatten each patch into a vector: $16 \times 16 \times 3$ (RGB) = 768 numbers 4. Project each 768-number vector into a 768-dimensional embedding (one linear layer) 5. Add positional encoding (so the model knows patch #1 is top-left, patch #196 is bottom-right) 6. Feed 196 patch embeddings into a standard Transformer 7. The Transformer produces 196 output embeddings — one per patch 8. Add a special [CLS] token at position 0 → its output embedding = the whole image representation

What the model learns: Without any explicit instruction, ViT learns to attend to semantically meaningful regions. Show it a dog → it attends to the dog's face and paws. Show it a shelf → it attends to the products, not the background.

Why it matters for VL-JEPA: - VL-JEPA uses a ViT as its Context Encoder AND Target Encoder - The 196 patch embeddings are exactly what gets masked, predicted, and compared - The [CLS] token embedding is what gets aligned with text descriptions

Key insight: The model has no built-in knowledge that pixels next to each other are related. It learns this from data. This makes it more flexible than CNNs — it can learn any spatial relationship, not just local ones.

Mental Model

```
Image (224x224 pixels)
↓
Cut into patches (16x16 each)
↓
[P1] [P2] [P3] ... [P196] ← 196 patches, like 196 words
↓
Each patch → embedding vector (768 numbers)
↓
Add position info: P1=top-left, P196=bottom-right
↓
Feed into Transformer (same as language Transformer)
↓
196 output embeddings — each patch now "knows" about all other patches
```

Paper 3: Learning Transferable Visual Models From Natural Language Supervision (CLIP, 2021)

Authors: Radford et al. (OpenAI) **Link:** <https://arxiv.org/abs/2103.00020> **Read:** Sections 1, 2, 3.1, 3.2, and Figure 1. Skip sections 4–5. **Time:** 45 minutes

Why You Need This

VL-JEPA's cross-modal alignment (connecting vision and language) is inspired directly by CLIP. CLIP showed the world that you can train a model to understand both images and text together, using only natural image-caption pairs from the internet — no manual labels needed.

Plain English Summary

The problem it solved: Training vision models traditionally required massive labeled datasets — ImageNet has 14 million images, each manually labeled by humans. This is expensive, slow, and limits what the model can recognize (only what's in the labels).

The big idea — Learn from image-text pairs: The internet is full of images with captions — news articles, product listings, social media, Wikipedia. What if you trained a model on 400 million such pairs? The caption is a free label — no human annotation needed.

The architecture: Two encoders trained together: - **Image Encoder** (ViT or ResNet): image → 512-dimensional vector - **Text Encoder** (Transformer): caption → 512-dimensional vector

The training objective — Contrastive Loss: For a batch of N image-text pairs: - Match each image to its correct caption → push their vectors close together - Push each image away from all N-1 wrong captions - Do the same for each text

After training on 400M pairs: the image of a dog and the text "a dog" end up as nearly identical vectors in 512-dimensional space.

What CLIP can do: - Zero-shot classification: never seen a "labrador" example → still classifies it correctly because it understands the word - Image search by text query - Text-to-image retrieval

Why it matters for VL-JEPA: VL-JEPA takes CLIP's cross-modal alignment idea + combines it with JEPA's predictive coding: - CLIP: align image embeddings with text embeddings (what CLIP does) - JEPA: predict masked image regions in embedding space (what JEPA adds) - VL-JEPA: do both at the same time on product images + descriptions

The key limitation of CLIP (why we need JEPA): CLIP is discriminative — it learns to match pairs, but doesn't build a deep understanding of the visual world. It can tell you "this image matches the text 'red chair'" but can't tell you "there are 3 chairs in this image." JEPA's predictive objective teaches deeper structure.

Mental Model

```
Training:
("Image of a cat") ↔ "a fluffy orange cat sitting on a mat"   ✓ Push together
("Image of a cat") ↔ "a blue sports car on a highway"        ✗ Push apart

After training:
encode("image of cat") ≈ encode("a fluffy orange cat")
encode("image of car") ≈ encode("a fast red sports car")

Application:
User types: "stainless steel shelf bracket"
→ encode the text
→ find nearest product image embeddings
→ return matching products ✓
```

Paper 4: Masked Autoencoders Are Scalable Vision Learners (MAE, 2021)

Authors: He et al. (Meta AI / Facebook AI Research) **Link:** <https://arxiv.org/abs/2111.06377> **Read:** Sections 1, 2, 3, and Figures 1-3. Skim section 4. **Time:** 30 minutes

Why You Need This

MAE is the direct predecessor to JEPA. Understanding MAE makes JEPA's innovation obvious. MAE masks image patches and reconstructs pixels — JEPA does the same but reconstructs *representations* instead of pixels. One change, massive impact.

Plain English Summary

The problem it solved: ViT showed Transformers can process images. But training ViT well requires either huge labeled datasets or complex self-supervised tricks. MAE showed a simpler approach: mask most of the image and make the model

reconstruct what's missing.

The big idea — Masked Autoencoding: 1. Take an image, split into 196 patches 2. Randomly mask 75% of patches (remove them entirely) 3. Show only 25% of patches to the encoder 4. A decoder tries to reconstruct the original pixel values of all masked patches 5. Loss = difference between predicted pixels and actual pixels

The model must learn rich visual representations to reconstruct masked regions. You can't reconstruct a hidden product image region without understanding what the product looks like.

Why masking 75% works: - Too little masking → easy task → model doesn't learn much - 75% masking → hard task → model must deeply understand structure - The model can't "cheat" by looking at nearby unmasked patches

The asymmetric encoder-decoder design: - **Encoder:** Only sees 25% of patches (the visible ones). This is the expensive part — large ViT. - **Decoder:** Sees all positions (visible + masked placeholders). Small, lightweight. - Result: 3× faster training because encoder only processes 25% of patches.

What MAE learns: Masking and reconstructing images teaches the model to understand: - Local texture (what does metal look like?) - Global structure (where would the handle of this tool be?) - Semantic context (this is a shelf, so there should be products here)

The limitation MAE has (why JEPA is better): MAE reconstructs raw pixels. Pixels are noisy, redundant, and low-level. Reconstructing the exact RGB value of each pixel wastes model capacity on irrelevant details (lighting, shadows, JPEG artifacts). What you really want to learn is: *what is this thing?* Not: *what is the exact color value of this pixel?*

This is exactly the insight that led to JEPA.

Comparison: MAE vs JEPA

```
MAE:
Masked patch → Reconstruct exact pixel values
Learning: "The pixel at position (47, 83) should be RGB(142, 98, 67)"
Problem: Wastes compute on irrelevant pixel details

JEPA:
Masked patch → Predict embedding (abstract representation)
Learning: "This region should have an embedding similar to: [0.3, -0.7, 0.2, ...]"
Benefit: Learns high-level concepts, ignores irrelevant pixel noise
```

Paper 5: Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA, 2023)

Authors: Assran et al. (Meta AI) **Link:** <https://arxiv.org/abs/2301.08243> **Read:** Full paper — it's only 15 pages and very well written. **Time:** 60–90 minutes

Why You Need This

This IS the architecture you're building (the image-only version). Every component of VL-JEPA starts here. This is the paper that matters most.

Plain English Summary

The problem it solved: Prior self-supervised methods fell into two categories: 1. **Invariance-based** (SimCLR, MoCo, DINO): train two augmented views of the same image to have similar embeddings. Problem: you have to carefully design augmentations, and the model learns to ignore image content (two augmented cats look the same = ignore the cat, only learn the augmentation). 2. **Generative** (MAE): reconstruct raw pixels. Problem: wastes capacity on low-level details.

JEPA offers a third way: predict representations of unseen regions, without augmentation tricks and without pixel reconstruction.

The core innovation — Predict in Representation Space:

```
Instead of:
masked region → reconstruct pixels      (MAE)
```

JEPA does:

context patches → predict EMBEDDING of masked patches (JEPA)

The embeddings come from a Target Encoder — a slowly-updating copy of the main encoder. The model must predict what the target encoder *would output* for the masked region, given only the context patches.

The four components:

- 1. Context Encoder (θ)** - A ViT that processes only the visible (unmasked) patches - Outputs context embeddings — rich representations of what it can see - Updated by gradient descent (learns actively)
- 2. Target Encoder (θ)** - Identical architecture to context encoder - Processes ALL patches (visible + masked) - Produces the "ground truth" target embeddings - Updated via Exponential Moving Average (EMA) — NOT by gradients - Update rule: $\theta \leftarrow m \times \theta + (1-m) \times \theta$, where $m \approx 0.996$
- 3. Predictor** - A small 3-layer Transformer that sits between context encoder and target encoder - Input: context embeddings + positional queries for masked patches - Output: predicted embeddings for each masked position - "Given what I can see, what should the masked region look like in embedding space?"

4. Loss Function

```
L = MSE(predicted_embeddings, target_embeddings)
    = mean((predictor(context_enc(visible)) - target_enc(masked))2)
```

Simple mean squared error between predicted and actual target embeddings. No contrastive loss. No augmentations. No negative pairs.

Why EMA prevents collapse: If both encoders were updated by gradients toward the same loss, they'd find the trivial solution: output all zeros (then loss = 0 forever). EMA prevents this: - Target encoder is updated slowly (momentum = 0.996 means it only changes 0.4% per step) - Context encoder updates fast - There's always a gap between them — the predictor is always chasing a moving target

The masking strategy: - Divide image into 196 patches (14×14 grid) - Select 4 random rectangular "target blocks" to mask (roughly 50% of patches) - Context = remaining visible patches - The model must predict the target blocks given the context
Rectangular blocks are harder than random patches — the model can't reconstruct a rectangular region just from adjacent context patches. It must understand the global content.

What I-JEPA learns: Without any labels, without any augmentations, trained only to predict masked representations: - Semantic understanding: similar objects cluster in embedding space - Spatial understanding: knows what "should" be at different positions - Transfer learning: fine-tunes well on ImageNet classification, object detection, semantic segmentation

Performance: - Matches or beats DINO, MAE, SimCLR on downstream tasks - Trains 2–3× faster than MAE (processes fewer patches, simpler loss) - Better semantic representations than generative methods

VL-JEPA Extension

VL-JEPA takes I-JEPA and adds:

- 1. Language Tower:** A text encoder (BERT-style) processes product descriptions → text embeddings
- 2. Cross-Modal Loss:** Pull image embeddings and matching text embeddings together in shared space (like CLIP)
- 3. Joint Training:** Both losses trained simultaneously
- 4. Predictive loss:** predict masked image regions (I-JEPA)
- 5. Alignment loss:** image ↔ text embedding alignment (CLIP-style)

Result: A model that understands BOTH visual content AND language descriptions, with deeper world understanding than pure CLIP.

Bonus Paper 1: CLIP (Deep Dive)

Full Title: Learning Transferable Visual Models From Natural Language Supervision **Link:** <https://arxiv.org/abs/2103.00020>
(Already summarized as Paper 3 — re-read section 3.2 for training details)

Bonus Paper 2: DINO / DINOv2 (2021/2023)

Authors: Caron et al. / Oquab et al. (Meta AI) **Link (DINOv2):** <https://arxiv.org/abs/2304.07193> **Read:** Abstract + Section 1 only **Time:** 15 minutes

Why It's Relevant

DINOv2 is the current state-of-the-art self-supervised vision model. It's what many production systems use today. Understanding it shows you where the field is and why JEPA is the next step.

Plain English Summary

DINO uses a teacher-student framework (similar to JEPA's EMA trick): - Student network sees augmented views - Teacher network (EMA) sees different augmented views - Train student to match teacher's output

DINOv2 scales this to 142M parameters trained on 142M curated images. The learned features are remarkably good at: - Semantic segmentation (without labels) - Depth estimation - Feature matching

JEPA vs DINO: - DINO requires augmentation engineering (hand-crafted transforms) - JEPA requires no augmentations — just masking - JEPA is more principled (LeCun's argument: augmentations encode human priors, not learned structure) - Practically: DINOv2 is stronger today, JEPA is the more theoretically sound direction

Reading Plan (2 Weeks)

Week 1: Foundations

Day	Paper	Focus
Mon	Paper 1 (Attention)	Read sections 1–3, draw the attention diagram
Tue	Paper 1 (Attention)	Implement attention from scratch in Python (no libraries)
Wed	Paper 2 (ViT)	Read sections 1–3, draw the patch embedding diagram
Thu	Paper 2 (ViT)	Sketch how a 224×224 image becomes 196 vectors
Fri	Paper 3 (CLIP)	Read sections 1–3.2, understand contrastive loss
Sat	Paper 3 (CLIP)	Think: how would you use CLIP for Lowe's product search?
Sun	Review + rest	Re-read anything unclear

Week 2: Self-Supervised Learning + JEPA

Day	Paper	Focus
Mon	Paper 4 (MAE)	Read sections 1–3, understand masking strategy
Tue	Paper 4 (MAE)	Draw MAE pipeline: image $\rightarrow$ mask $\rightarrow$ reconstruct
Wed	Paper 5 (I-JEPA)	Read full paper — take notes on each component
Thu	Paper 5 (I-JEPA)	Draw I-JEPA pipeline: context encoder $\rightarrow$ predictor $\rightarrow$ target encoder
Fri	Paper 5 (I-JEPA)	Write in your own words: why is JEPA better than MAE?
Sat	Bonus: DINOv2 abstract	Understand the competitive landscape
Sun	Review + questions	Write down every question you have for me

Key Concepts Cheat Sheet

Concept	One-Line Definition
Embedding	A list of numbers representing the meaning of something
Attention	Mechanism for each element to "look at" all others and decide what matters
Transformer	Neural network built on attention, processes sequences in parallel
ViT	Vision Transformer — treats image patches as a sequence
Self-supervised learning	Training without human labels — the data labels itself
Contrastive learning	Pull matching pairs together, push non-matching apart
MAE	Mask image patches, reconstruct the missing pixels
JEPA	Mask image patches, predict the missing representations (not pixels)
EMA	Exponential Moving Average — slow-update trick to prevent model collapse
Predictor	Small network that bridges context encoder and target encoder
Cross-modal alignment	Making image and text embeddings live in the same space
VL-JEPA	JEPA + language tower + cross-modal alignment
k-NN	k-Nearest Neighbors — find the k most similar embeddings in a database
Zero-shot	Model does a task it was never explicitly trained for
Fine-tuning	Take a pretrained model, continue training on a smaller specific dataset

After Reading: What You Should Be Able to Explain

By the end of Week 2, you should be able to answer these without looking anything up:

1. Why does JEPA predict in representation space instead of pixel space?
2. What is the EMA target encoder and why is it necessary?
3. How does CLIP connect images and text?
4. What is a patch embedding and how is it computed?
5. How is self-attention different from regular (dense) attention?
6. Why does MAE mask 75% of patches but I-JEPA uses rectangular blocks?
7. What is the difference between discriminative and generative models?
8. Why is VL-JEPA better than CLIP for product understanding?

Next Step After Reading

When you've finished these papers, message me and we'll start building. I'll have the first Colab notebook ready:

"Week 1: Building Multi-Head Self-Attention from Scratch in PyTorch" - 200 lines of code - Every line explained - Run it in Google Colab — free, no GPU needed for Week 1 - Input: a sentence / Output: attention weights visualized

You'll write the foundation of every AI model that exists today — from scratch. That's where we start.

